P07 – Le Refuge

Identification de races de chiens par Deep Learning avec des modèles datant de moins de 5 ans

Projet POC – Streamlit & Deep Learning | Dataset : Stanford Dogs (120 races, 20 580 images)

Sommaire

01 **Contexte & dataset**

Objectifs du POC · Stanford Dogs · EDA · Problématique

02 **Architectures Deep Learning**

Évolution CNN → Transformers · MobileNetV2 · ConvNeXt · DINOv3 · Justification des choix

03 **Modélisation & entraînement**

Augmentations · Stratégie fine-tuning · Courbes d'apprentissage · Artefacts HuggingFace

04 **Dashboard Streamlit**

Interface & navigation · Onglet identification · Comparaison des performances · Analyse des résultats

05 **Synthèse & perspectives**

Bilan du POC · Pistes d'amélioration · Déploiement

Contexte et objectifs

Contexte métier

- Un refuge canin doit identifier la race d'un chien à partir d'une photo
- Identification manuelle : lente, subjective, expertise requise
- Besoin d'un outil simple, rapide, accessible aux bénévoles

Objectifs du POC

- Construire une application web Streamlit utilisable par tous
- Tester et comparer plusieurs architectures Deep Learning
- Évaluer précision, vitesse d'inférence et poids des modèles
- Déployer l'application permettant de choisir son modèle et de lancer les prédictions sur le cloud via streamlit cloud.

Approche : Transfer Learning sur 3 architectures (MobileNetV2 baseline · ConvNeXt-Tiny · DINOv3 ViT-B/16)

2 modèles récents (< 5 ans) sur un dataset complet

Jeu de données : Stanford Dogs

120
races

20 580
images

70/15/15
train / val / test

Préparation

- Split stratifié 70 % train / 15 % val / 15 % test
- Contrôle de la distribution d'images par race
- Analyse des dimensions (largeur / hauteur)

Challenges

- Forte variabilité : postures, luminosité, arrière-plans
- Races visuellement très proches (ex. Malamute vs Husky)
- Nombre d'images inégal selon les races

Analyse exploratoire (EDA)

Statistiques globales

- Nombre total d'images et de races
- Nombre moyen / médian d'images par race
- Distribution : min 148 — max 252 images / race

Visualisations dans l'app

- Histogramme : distribution du nb d'images par race
- Boxplots : largeur et hauteur des images
- Galerie : exemples d'images par race sélectionnée

Observations clés

- Dataset relativement équilibré → pas de sur-représentation majeure à corriger
- Tailles d'images très variables → redimensionnement systématique en 224×224 nécessaire
- Forte diversité visuelle intra-classe → modèles robustes requis

Problématique : pourquoi la classification de races est difficile

Variabilité intra-classe

- Une même race peut avoir des morphologies très différentes
- Les couleurs de pelage sont très variables
- Les postures et angles de prise de vue également

Similarité inter-classes

- Races proches génétiquement
- Distinction sur des détails fins (oreilles, museau, taille)
- Ex : Malamute vs Husky Sibérien

Conditions réelles

- Images prises au smartphone
- Arrière-plans complexes
- Luminosité et netteté aléatoires

Ces défis justifient le recours à des architectures récentes et puissantes, entraînées sur des milliards d'images, capables d'extraire des représentations visuelles très fines. Ici nous allons tenter de voir si des modèles plus récents peuvent améliorer le taux de prédcitions.

Partie 1 – Architectures de Deep Learning

Du CNN classique aux modèles de vision modernes

Évolution des architectures de vision par ordinateur

2012–2019
CNN classiques

AlexNet, VGG, ResNet

- Convolutions locales empilées pour extraire des caractéristiques
- Couches de pooling pour réduire la dimensionnalité
- Couches entièrement connectées (FC) pour la classification finale

2019–2021
MobileNetV2

Baseline légère

- Depthwise Separable Convolutions pour réduire considérablement le nombre de calculs
- Blocs Inverted Residuals pour une meilleure efficacité sur mobile
- Compromis précision / vitesse

2021
ConvNeXt

CVPR 2022

- Modernise l'architecture CNN en s'inspirant des Vision Transformers (ViT)
- Grands noyaux 7×7, LayerNorm, GELU et structure par étapes
- Comble l'écart de performance avec les ViT

2021–2023
Vision Transformers + DINOv3

Oquab et al. 2023

- Architecture basée sur l'attention (Self-Attention) traitant l'image comme une séquence de patches
- DINOv3 pré-entraîné en auto-supervisé sur 142 M images
- Représentations visuelles robustes et universelles

Notre choix pour le projet : **MobileNetV2** (baseline) · **ConvNeXt-Tiny** (CNN moderne) · **DINOv3 ViT-B/16** (Transformer self-supervised)
3 générations, 3 niveaux de complexité pour une analyse comparative complète.

MobileNetV2 — La baseline légère

Principe

- CNN optimisé pour les appareils mobiles (2018, Google)
- Depthwise Separable Convolutions : sépare les filtres spatiaux des filtres de profondeur
- Réduit massivement le nombre d'opérations (~8–9×)
- Inverted Residuals : les blocs qui élargissent puis rétrécissent les canaux

Avantages & limites

- Très léger : environ 3,4 M paramètres
- Inférence rapide, adapté au mobile / embarqué
- Précision limitée sur tâches fines (76 % Top-1)
- Capacité de représentation inférieure aux modèles modernes

Architecture simplifiée

Rôle dans le POC : représente la génération CNN efficiente. Sert de référence pour mesurer le gain apporté par ConvNeXt et DINOv3.

Pourquoi tester au-delà des CNN classiques ?

Limites des CNN classiques

- Champ réceptif local : chaque filtre ne voit qu'un voisinage limité
- Relations longue distance difficiles à capturer
- Design figé : même convolution 3×3 répétée
- Moins efficaces sur datasets complexes à grande échelle
- Performances plafonnées sur classifications fines (120 races)

Ce qu'il faut pour 120 races de chiens

→ **Représentations globales**

Comprendre la forme entière du chien, pas juste des patches

→ **Détails fins**

Distinguer oreilles, museau, texture de pelage entre races proches

→ **Robustesse**

Fonctionner malgré la variabilité de poses, fonds, éclairages

→ **Pré-entraînement riche**

Apprendre sur des millions d'images avant le fine-tuning

ConvNeXt modernise le CNN pour répondre à ces besoins
DINOv3 va encore plus loin grâce au self-supervised learning à très grande échelle

ConvNeXt-Tiny — Un CNN modernisé (Liu et al., 2022)

L'idée fondatrice

- Partir d'un ResNet-50 et appliquer une à une les innovations des Vision Transformers
- Résultat : un CNN pur qui surpasse Swin Transformer sur ImageNet
- Démontre que les CNN ne sont pas "dépassés" , ils manquaient juste de modernisation
- Publication CVPR 2022 — rapidement adopté en production

Design choices clés

Patchify Stem

Conv 4×4 stride 4 en entrée (comme ViT)

Depthwise Conv 7×7

Champ réceptif plus large (vs 3×3 classique)

Inverted Bottleneck

Canaux larges dans les couches internes

LayerNorm

Remplace BatchNorm pour la stabilité

GELU activation

Remplace ReLU (comme dans les Transformers)

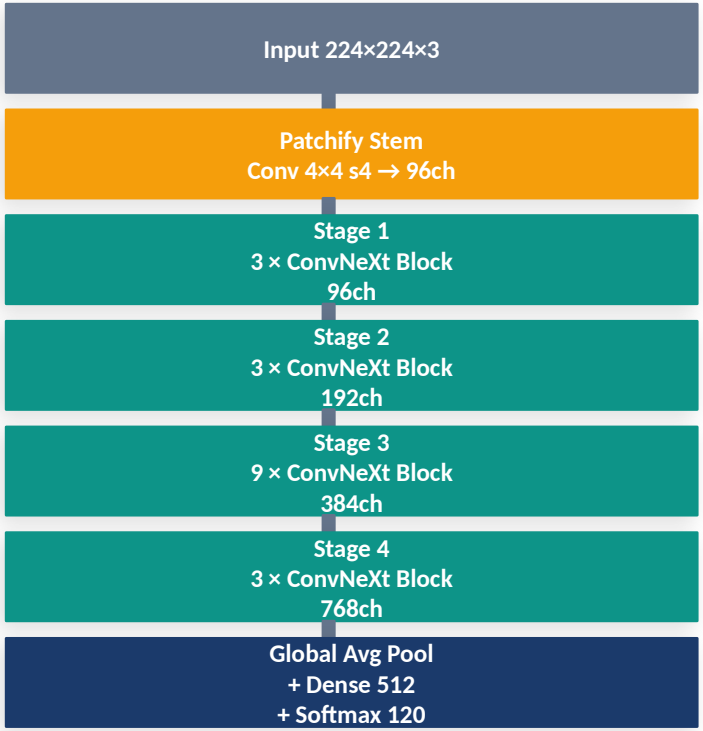
Fewer activations

Design épuré, moins de non-linéarités parasites

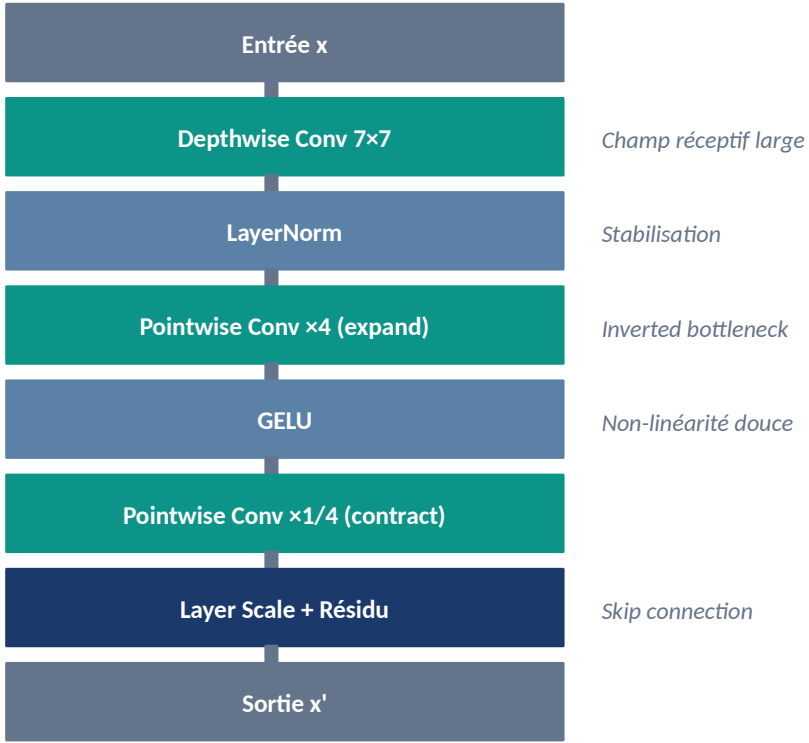
Résultat : ConvNeXt-Tiny obtient 82,1 % Top-1 sur ImageNet-1K — comparable aux Transformers, mais plus simple à déployer et plus rapide à l'inférence.

ConvNeXt-Tiny — Architecture & bloc ConvNeXt

Architecture globale



Bloc ConvNeXt (détail)



DINOv3 — Self-supervised Vision Transformer (Oquab et al., 2023)

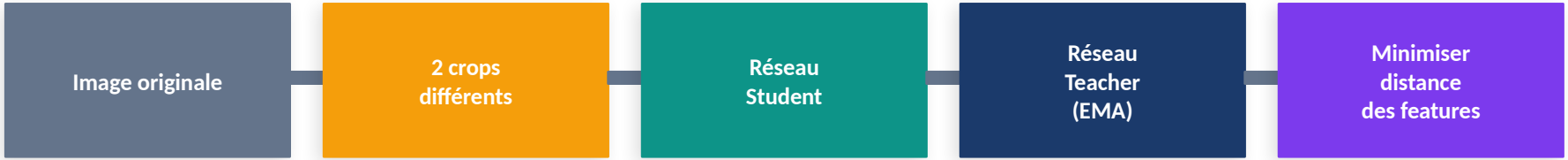
Qu'est-ce que DINOv3 ?

- Modèle de Meta AI entraîné sans étiquettes sur 142 millions d'images (LVD-142M)
- Self-supervised : apprend à représenter les images sans supervision humaine
- Backbone ViT-B/16 : Vision Transformer base, patches de 16×16 pixels
- Produit des features universelles : utilisables sur n'importe quelle tâche visuelle

Pourquoi c'est puissant ?

- Entraîné sur 142 M images → représentations extrêmement riches
- Zéro annotation nécessaire → scalable à l'infini
- Features immédiatement transférables (classification, segmentation, détection)
- Surpasse des modèles supervisés sur de nombreux benchmarks

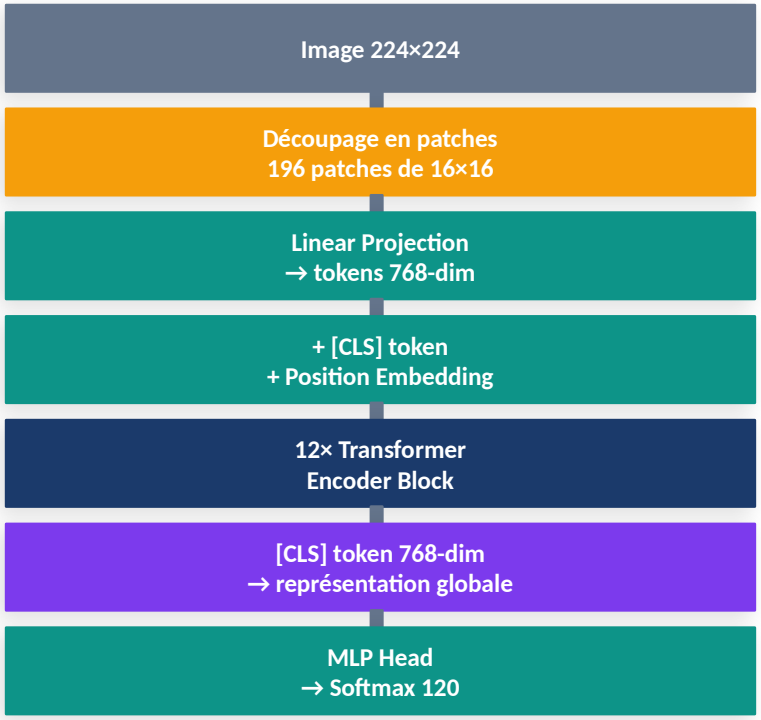
Principe du self-supervised learning (DINO)



Sans aucune étiquette → le modèle apprend à produire les mêmes représentations pour deux vues différentes de la même image

DINOv3 — Architecture Vision Transformer (ViT-B/16)

Pipeline ViT-B/16



Bloc Transformer Encoder (×12)

Justification du choix des modèles pour ce POC

Critère	MobileNetV2	ConvNeXt-Tiny	DINOv3 ViT-B/16
Pré-entraînement	ImageNet-1K	ImageNet-1K (+ 22K)	LVD-142M (self-sup.)
Capacité repr.	Faible	**Haute**	Très haute
Champ réceptif	Local 3×3	**Élargi 7×7**	Global (attention)
Nb paramètres	~3,4 M	**~28 M**	~86 M
Fine-tuning	Facile	**Facile**	Exige plus de RAM
Adapté 120 races	Limite	**Oui**	Oui

ConvNeXt-Tiny : meilleur compromis
Précision élevée · déploiement simple · inférence rapide

DINOv3 : features universelles ultra-riches
Pré-entraînement massif → robustesse maximale

Partie 2 – Modélisation & entraînement

Fine-tuning, augmentations et stratégie d'apprentissage

Pré-traitements & augmentations de données

Resize
224×224

Égalisation
histogramme

Flou
gaussien

Détection
contours

Augmentations — ConvNeXt-Tiny

- RandomRotation ±20°
- Width / Height shift 0.2
- Shear range 0.15 · Zoom range 0.15
- Horizontal flip
- Fill mode : nearest

Augmentations — DINOv3 ViT-B/16

- RandomResizedCrop 224×224 (scale 0.7–1.0)
- RandomRotation ±20°
- Horizontal flip
- ColorJitter : brightness/contrast/saturation 0.2, hue 0.1
- Normalisation ImageNet (mean/std)

Stratégie d'entraînement & fine-tuning

MobileNetV2

- Backbone entièrement gelé (ImageNet weights)
- Ajout Global Avg Pool + Dense 512 + Dropout 0.5
- Entraînement tête seule uniquement — 50 epochs max
- EarlyStopping patience=10 (val_loss)
- ReduceLROnPlateau factor=0.5, patience=5
- Optimizer : Adam, lr=1e-3

ConvNeXt-Tiny

- Backbone gelé (ImageNet weights)
- Ajout GAP + Dense 512 + Dropout 0.5+0.3 + Softmax 120
- Phase 1 : tête seule, AdamW lr=5e-4, 20 epochs
- Phase 2 : dégel partiel backbone, lr=1e-4, 50 epochs max
- EarlyStopping patience=15, ReduceLROnPlateau patience=5

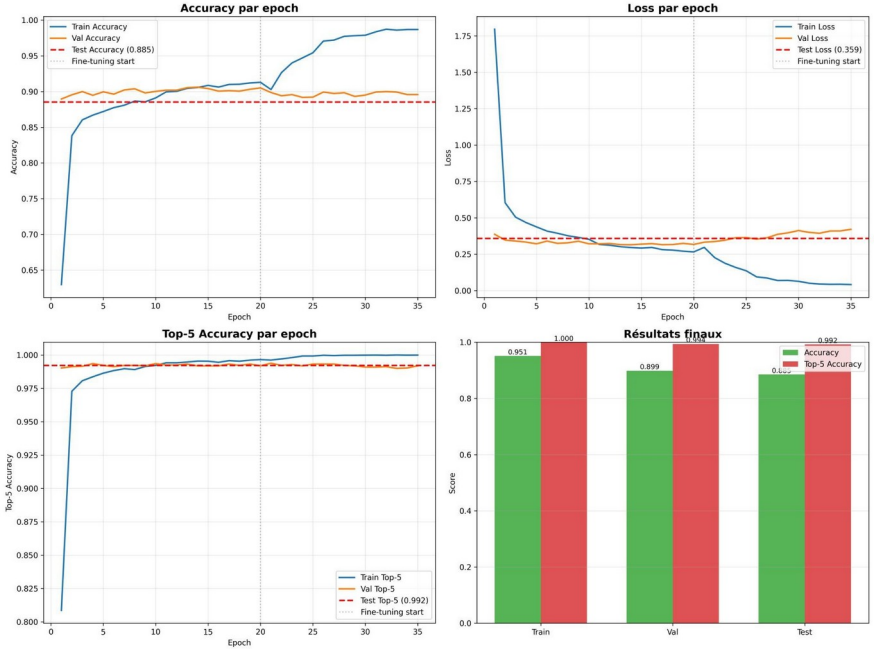
DINOv3 ViT-B/16

- Backbone gelé (LVD-142M weights .pt)
- Ajout MLP : Dropout 0.5+0.3 + Linear 768→120
- Phase 1 : tête seule, AdamW lr=5e-4, 20 epochs
- Phase 2 : 6 derniers blocs dégelés, lr=1e-4, 50 epochs max
- Early stopping patience=15, CosineAnnealing eta_min=1e-7

Fonction de perte : Categorical Crossentropy | Métrique principale : Top-1 Accuracy | Évaluation : Top-5 Accuracy

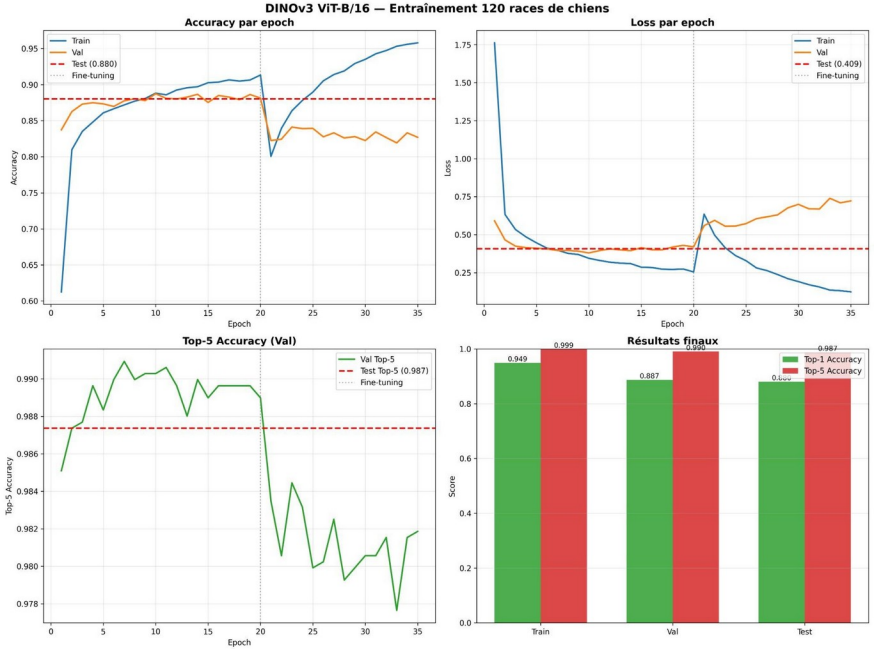
Courbes d'apprentissage

ConvNeXt-Tiny



Convergence propre · Fine-tuning (ep. 20) améliore le train sans dégrader la val · Pas d'overfitting majeur

DINOv3 ViT-B/16



Instabilité val à ep. 20 (dégel des blocs) · La val loss remonte → early stopping déclenché · Train continue à progresser

Gestion des modèles & artefacts

Stockage

- Poids Keras : .h5
- Poids PyTorch : .pt
- Encodeurs labels : .pkl
- Hébergés sur HuggingFace Hub
- Dépôt : LudGold/P07_POC

Chargement dans l'app

- Téléchargement au 1er lancement
- Gestion des erreurs réseau
- Indicateur de statut sidebar
- Hebergée sur le cloud de streamlit

Avantages

- Application portable
- Séparation code / poids
- Mise à jour modèle sans déployer de nouveau
- Versioning HuggingFace

Partie 3 – Dashboard Streamlit

Interface déployée sur le cloud, navigation et expérience utilisateur

Interface Streamlit — Structure & navigation

Exploration des données

Onglet 1

Stats globales, histogramme distribution, boxplots dimensions, galerie d'images par race

Identification (prédiction)

Onglet 2

Choix du modèle, upload photo, lancement identification, affichage résultats Top-1 & Top-5

Performance des modèles

Onglet 3

Tableau récapitulatif, accuracy, paramètres, temps d'inférence, modèle retenu

Barre latérale (sidebar)

- Présentation des 3 modèles disponibles
- Statut de chargement (✓ chargé / ✗ erreur)
- Rappel du dataset et du split utilisé

Accessibilité - Normes mises en place pour le front de l'app

- Palette WCAG 2.1 (contrastes suffisants)
- Titres, métriques et légendes Plotly lisibles
- Design simple : aucune expertise technique requise

Onglet Identification — Flux utilisateur & résultats

1
Choisir un modèle
ConvNeXt, MobileNet ou DINOv3

2
Charger une photo
JPG, JPEG ou PNG

3
Lancer l'identification
Clic sur le bouton

Résultats affichés

- Race prédite Top-1 : nom formaté (ex. « Golden Retriever »)
- Confiance en % : message succès si ≥ 50 %, avertissement si faible
- Temps d'inférence affiché en millisecondes
- Top-5 prédictions : barres de progression avec race + score de confiance
- Gestion des cas limites : image illisible, race non présente dans la distribution → message explicite

Comparaison des performances — Modèle retenu : ConvNeXt-Tiny

Jeu de test : 3 087 images — 119 races | Phase 1 : 20 epochs · Phase 2 : 15 epochs (early stopping)

Modèle	Top-1 Acc.	Top-5 Acc.	Params (M)	Inférence
ConvNeXt-Tiny	**88,53 %**	**99,22 %**	**~28 M**	**Modérée**
MobileNetV2	~76 %	~96 %	~3,4 M	Rapide
DINOv3 ViT-B/16	88,01 %	98,74 %	~86 M	Lente

Modèle retenu : ConvNeXt-Tiny

+12,5 pts vs MobileNetV2 sur Top-1 · **Résultats équivalents à DINOv3** avec 3× moins de paramètres · **Inférence plus rapide** → meilleur compromis pour un déploiement réel

Analyse des résultats — Forces et limites

Forces

- 88,5 % Top-1 sur 120 races : performance solide pour un POC
- Top-5 99,2 % : la bonne race est presque toujours dans les 5 premières
- Fine-tuning efficace : convergence rapide grâce aux poids ImageNet
- Interface Streamlit accessible : aucune compétence technique requise

Limites identifiées

- 11,5 % d'erreurs Top-1 : races très proches encore confondues
- DINOv3 entraîné tête seule : fine-tuning complet améliorerait les scores
- Images en conditions réelles (smartphone) non testées
- Pas d'historisation des prédictions
- Dataset 100 % occidental : nécessiterait des photos du monde entier

Pistes d'amélioration

- Fine-tuning complet de DINOv3 avec un GPU plus puissant pourrait apporter un gain estimé de +2 à +4 pts
- Données supplémentaires : scraping d'images réelles de refuges pour mieux couvrir les conditions terrain
- ConvNeXt + DINOv3 : Combiner les deux modèles pour réduire les erreurs sur les races trop proches
- Seuil de confiance adaptatif : renvoyer "race incertaine" plutôt qu'une réponse erronée si confiance < 30 %

Synthèse & perspectives

Bilan du POC

- Faisabilité démontrée : 88,5 % Top-1 sur 120 races
- ConvNeXt-Tiny retenu : meilleur compromis précision / coût
- Architecture moderne validée pour la classification fine
- Interface Streamlit simple, accessible aux non-experts
- Pipeline complet : EDA → modélisation → app → comparaison

Perspectives

- Déploiement sur serveur interne ou cloud (Hugging Face Spaces)
- Fine-tuning complet DINOv3 pour maximiser les performances
- Base de données pour historiser et tracer les prédictions
- Optimisation mobile / tablette pour usage en refuge
- Extension à d'autres animaux (chats, lapins…)

FIN